

Genetic Algorithm for Blackjack

Pseudocode & Neural Network Architecture

Students: Simão Pereira n901396; Filipa Delfino n901397.

1. Algorithm Overview – Main GA and Analysis Pipeline

A single evolutionary algorithm ("main") was first built to run and validate the Blackjack environment, evolve a population of neural-network agents, and compare best/average fitness curves, including a side-by-side tournament-vs-rank comparison mode. To draw a more robust conclusion about which combination of operators yields the agent with the highest real win rate, an "analyze" pipeline was then built on top of it: it repeats full 100-generation evolutions for every selection/crossover/mutation combination, evaluates the resulting best individual over 500 games, and repeats this 5 times per combination to average out randomness. Both procedures are summarized step-by-step below.

Algorithm – Genetic Blackjack Agent (Main + Analysis Pipeline)

STEP-BY-STEP DESCRIPTION

PART A, Main algorithm (developed first, to validate the environment and compare fitness/operator behaviour)

1. Initialize the Blackjack environment (Gymnasium Blackjack-v1) and create a population of 50 individuals; each individual is an 81-gene vector encoding the weights of a 3-16-1 neural network.
2. Repeat for 100 generations:
 - 2.1 Evaluate every individual's fitness by decoding it into a neural network and averaging its reward over N_GAMES of Blackjack.
 - 2.2 Record the generation's max and average fitness (for the convergence plots) and keep the best individual seen so far (elitism).
 - 2.3 Refill the population: keep the best individual unchanged, then repeatedly (a) select two parents with the chosen selection **method (Tournament or Rank)**, (b) **recombine them with the chosen crossover method (Uniform, Single-Point or Two-Point)**, and (c) mutate the offspring at the chosen mutation rate, until the *population is full again*.
3. Return the best individual found, its fitness, and the max/average fitness history. The main script can launch this evolution with both `SELECTION_METHOD = tournament` and `= rank` at the same time, plotting both curves together so the two strategies can be compared directly.

PART B, Analysis pipeline (built on top of the main algorithm to find, with statistical support, the sequence of operators that wins the most games)

4. For every combination of selection method (tournament, rank), crossover method (uniform, single-point, two-point) and mutation rate (1%, 10%):
 - 4.1 Run the main algorithm (steps 1-3) for 100 generations to obtain that combination's best-trained individual.
 - 4.2 Evaluate this individual's real win rate by playing 500 games of Blackjack.
 - 4.3 Repeat steps 4.1-4.2 five times for the same combination, so the win rate can be summarized as a mean and a standard deviation instead of a single noisy run.
5. Rank every combination by its mean win rate and plot the results with error bars (standard deviation) to identify the most reliable selection/crossover/mutation sequence.

2. Subroutines

The subroutines below are called by the main algorithm. Selection methods implement two distinct strategies: Tournament Selection uses pressure-based competition among k candidates, while Rank Selection avoids issues with negative fitness by assigning selection probability proportional to the individual's rank in the sorted population. Three crossover operators are provided: Uniform (gene-by-gene coin flip), Single-Point (one random cut), and Two-Point (two random cuts with alternating segments).

Subroutines - Operator Definitions

STEP-BY-STEP DESCRIPTION

1. `CREATE_POPULATION` - builds the initial population by drawing each of the 81 genes of every individual independently from a Uniform (-1, 1) distribution.
2. `EVALUATE_FITNESS` - decodes an individual into its 3→16→1 neural network, plays a number of Blackjack games by choosing Hit/Stand from the network output at every step, and returns the average reward per game as the individual's fitness.
3. `TOURNAMENT_SELECT` - picks a parent by randomly sampling k = 3 individuals from the population (without replacement) and returning the one with the highest fitness; selection pressure comes from local competition between the sampled candidates.
4. `RANK_SELECT` - picks a parent without depending on the sign or scale of the fitness values: the population is sorted by fitness, each individual receives a selection probability proportional to its rank (1 = worst, N = best), and a parent is drawn from that probability distribution.
5. `UNIFORM_CROSSOVER` - builds a child by choosing each gene independently from either parent with equal (50%) probability.
6. `SINGLE_POINT_CROSSOVER` - draws one random cut point and builds the child by taking all genes before the cut from parent 1 and all genes from the cut onward from parent 2.
7. `TWO_POINT_CROSSOVER` - draws two random cut points and builds the child from parent 1's genes before the first cut, followed by parent 2's genes from the first cut to the end (covering both the middle and last segments).

```
8. MUTATE - visits every gene of the child and, with probability equal
to the mutation rate, adds Gaussian noise (mean 0, std 0.5) to it,
then clips the result to the [-5, 5] range; genes not selected are
left unchanged.
```

3. Neural Network Architecture

Each individual in the population encodes a fully-connected neural network with 3 input neurons, 1 hidden layer of 16 neurons (ReLU activation), and 1 output neuron (Sigmoid activation). The 81-gene DNA vector is decoded into weight matrices and bias vectors as shown below.

Layer	Neurons	Activation	DNA indices
Input	3	-	-
W1 (Input→Hidden)	$3 \times 16 = 48$ weights	-	0 ... 47
b1 (Hidden bias)	16	-	48 ... 63
Hidden	16	ReLU	-
W2 (Hidden→Output)	$16 \times 1 = 16$ weights	-	64 ... 79
b2 (Output bias)	1	-	80
Output	1	Sigmoid	-

3.1 DNA Layout and Forward Pass

DNA Layout & Forward Pass	
DNA vector - 81 real-valued genes $\in [-5, 5]$	
Index 0 ... 47	→ W1 (48 weights, reshaped to 3×16)
Index 48 ... 63	→ b1 (16 biases, hidden layer)
Index 64 ... 79	→ W2 (16 weights, reshaped to 16×1)
Index 80	→ b2 (1 bias, output neuron)
Forward pass:	
$z1 = x \cdot W1 + b1$	// $x = (\text{player_sum}, \text{dealer_card}, \text{usable_ace})$
$a1 = \text{ReLU}(z1)$	// $\max(0, z1)$
$z2 = a1 \cdot W2 + b2$	
$a2 = \text{Sigmoid}(z2)$	// $1 / (1 + e^{(-z2)}) \in (0, 1)$
action = 1 (Hit) IF $a2 > 0.5$ ELSE 0 (Stand)	

3.2 Input Encoding

The Blackjack environment (Gymnasium Blackjack-v1) returns a 3-tuple observation at each step: (player_sum $\in [4, 21]$, dealer_card $\in [1, 10]$, usable_ace $\in \{0, 1\}$). These three values are cast to float and fed directly into the input layer. No normalisation is applied; the GA evolves weights that are robust to the raw scale of the inputs.

3.3 Output Decoding

The single Sigmoid output is thresholded at 0.5: values above 0.5 produce action 1 (Hit), values at or below 0.5 produce action 0 (Stand). This binary mapping directly corresponds to the two legal actions in standard Blackjack.